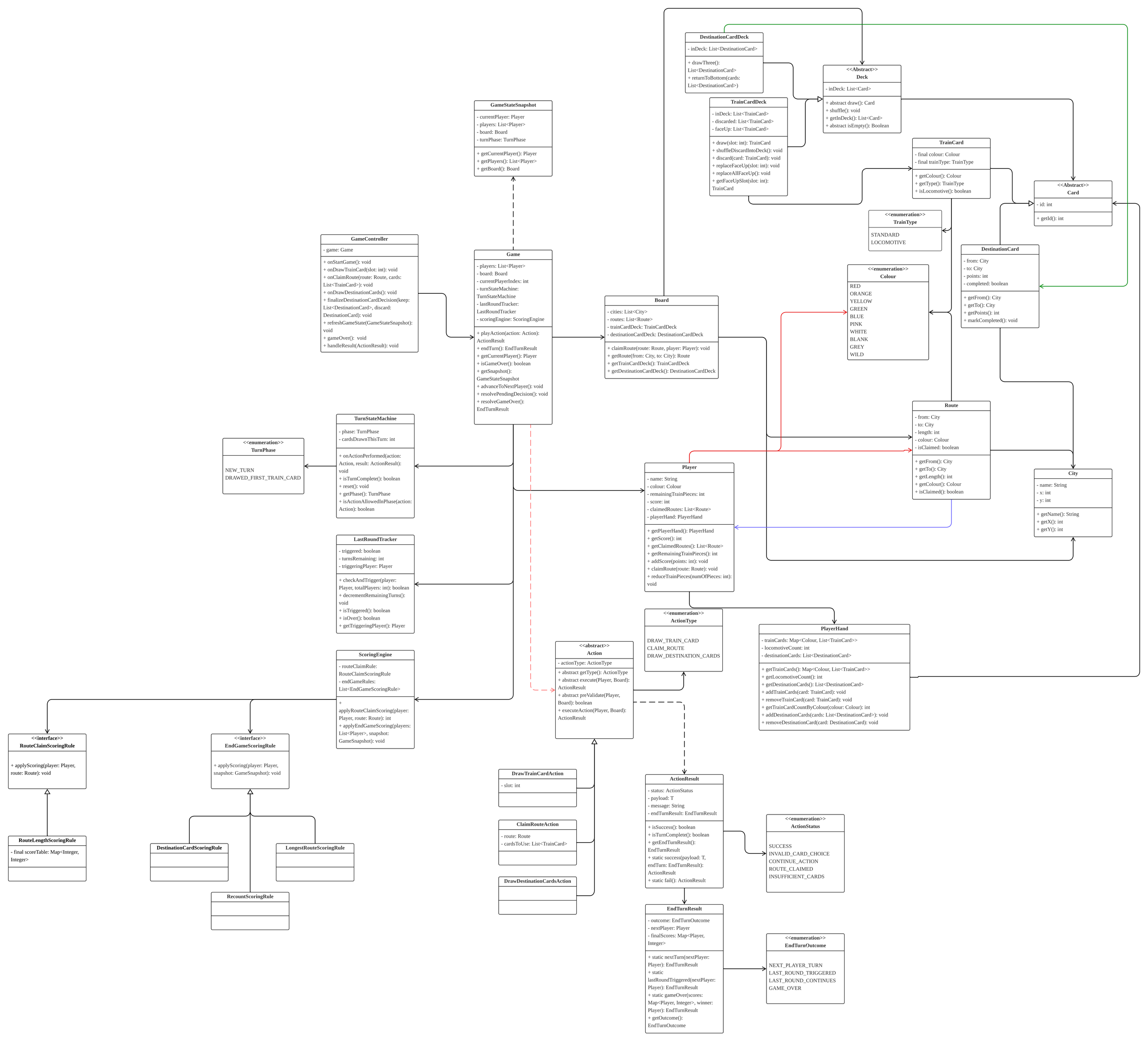

DestinationCardDeck
- inDeck: List<DestinationCard>
+ drawThree(): List<DestinationCard>
+ returnToBottom(cards: List<DestinationCard>)
<<Abstract>> Deck
- inDeck: List<Card>
+ abstract draw(): Card
+ shuffle(): void
+ getInDeck(): List<Card>
+ abstract isEmpty(): Boolean
TrainCardDeck
- inDeck: List<TrainCard>
- discarded: List<TrainCard>
- faceUp: List<TrainCard>
+ draw(slot: int): TrainCard
+ shuffleDiscardIntoDeck(): void
+ discard(card: TrainCard): void
+ replaceFaceUp(slot: int): void
+ replaceAllFaceUp(): void
+ getFaceUpSlot(slot: int): TrainCard
TrainCard
- final colour: Colour
- final trainType: TrainType
+ getColour(): Colour
+ getType(): TrainType
+ isLocomotive(): boolean
<<Abstract>> Card
- id: int
+ getId(): int
<<enumeration>> TrainType
STANDARD
LOCOMOTIVE
DestinationCard
- from: City
- to: City
- points: int
- completed: boolean
+ getFrom(): City
+ getTo(): City
+ getPoints(): int
+ markCompleted(): void
<<enumeration>> Colour
RED
ORANGE
YELLOW
GREEN
BLUE
PINK
WHITE
BLANK
GREY
WILD
GameStateSnapshot
- currentPlayer: Player
- players: List<Player>
- board: Board
- turnPhase: TurnPhase
+ getCurrentPlayer(): Player
+ getPlayers(): List<Player>
+ getBoard(): Board
GameController
- game: Game
+ onStartGame(): void
+ onDrawTrainCard(slot: int): void
+ onClaimRoute(route: Route, cards: List<TrainCard>): void
+ onDrawDestinationCards(): void
+ finalizeDestinationCardDecision(keep: List<DestinationCard>, discard: DestinationCard): void
+ refreshGameState(GameStateSnapshot): void
+ gameOver(): void
+ handleResult(ActionResult): void
Game
- players: List<Player>
- board: Board
- currentPlayerIndex: int
- turnStateMachine: TurnStateMachine
- lastRoundTracker: LastRoundTracker
- scoringEngine: ScoringEngine
+ playAction(action: Action): ActionResult
+ endTurn(): EndTurnResult
+ getCurrentPlayer(): Player
+ isGameOver(): boolean
+ getSnapshot(): GameStateSnapshot
+ advanceToNextPlayer(): void
+ resolvePendingDecision(): void
+ resolveGameOver(): EndTurnResult
Board
- cities: List<City>
- routes: List<Route>
- trainCardDeck: TrainCardDeck
- destinationCardDeck: DestinationCardDeck
+ claimRoute(route: Route, player: Player): void
+ getRoute(from: City, to: City): Route
+ getTrainCardDeck(): TrainCardDeck
+ getDestinationCardDeck(): DestinationCardDeck
Route
- from: City
- to: City
- length: int
- colour: Colour
- isClaimed: boolean
+ getFrom(): City
+ getTo(): City
+ getLength(): int
+ getColour(): Colour
+ isClaimed(): boolean
City
- name: String
- x: int
- y: int
+ getName(): String
+ getX(): int
+ getY(): int
TurnStateMachine
- phase: TurnPhase
- cardsDrawnThisTurn: int
+ onActionPerformed(action: Action, result: ActionResult): void
+ isTurnComplete(): boolean
+ reset(): void
+ getPhase(): TurnPhase
+ isActionAllowedInPhase(action: Action): boolean
<<enumeration>> TurnPhase
NEW_TURN
DRAWED_FIRST_TRAIN_CARD
Player
- name: String
- colour: Colour
- remainingTrainPieces: int
- score: int
- claimedRoutes: List<Route>
- playerHand: PlayerHand
+ getPlayerHand(): PlayerHand
+ getScore(): int
+ getClaimedRoutes(): List<Route>
+ getRemainingTrainPieces(): int
+ addScore(points: int): void
+ claimRoute(route: Route): void
+ reduceTrainPieces(numOfPieces: int): void
LastRoundTracker
- triggered: boolean
- turnsRemaining: int
- triggeringPlayer: Player
+ checkAndTrigger(player: Player, totalPlayers: int): boolean
+ decrementRemainingTurns(): void
+ isTriggered(): boolean
+ isOver(): boolean
+ getTriggeringPlayer(): Player
ScoringEngine
- routeClaimRule: RouteClaimScoringRule
- endGameRules: List<EndGameScoringRule>
+ applyRouteClaimScoring(player: Player, route: Route): int
+ applyEndGameScoring(players: List<Player>, snapshot: GameSnapshot): void
<<abstract>> Action
- actionType: ActionType
+ abstract getType(): ActionType
+ abstract execute(Player, Board): ActionResult
+ abstract preValidate(Player, Board): boolean
+ executeAction(Player, Board): ActionResult
<<enumeration>> ActionType
DRAW_TRAIN_CARD
CLAIM_ROUTE
DRAW_DESTINATION_CARDS
PlayerHand
- trainCards: Map<Colour, List<TrainCard>>
- locomotiveCount: int
- destinationCards: List<DestinationCard>
+ getTrainCards(): Map<Colour, List<TrainCard>>
+ getLocomotiveCount(): int
+ getDestinationCards(): List<DestinationCard>
+ addTrainCards(card: TrainCard): void
+ removeTrainCard(card: TrainCard): void
+ getTrainCardCountByColour(colour: Colour): int
+ addDestinationCards(cards: List<DestinationCard>): void
+ removeDestinationCard(card: DestinationCard): void
<<interface>> RouteClaimScoringRule
+ applyScoring(player: Player, route: Route): void
<<interface>> EndGameScoringRule
+ applyScoring(player: Player, snapshot: GameSnapshot): void
RouteLengthScoringRule
- final scoreTable: Map<Integer, Integer>
DestinationCardScoringRule
LongestRouteScoringRule
RecountScoringRule
DrawTrainCardAction
- slot: int
ClaimRouteAction
- route: Route
- cardsToUse: List<TrainCard>
DrawDestinationCardsAction
ActionResult
- status: ActionStatus
- payload: T
- message: String
- endTurnResult: EndTurnResult
+ isSuccess(): boolean
+ isTurnComplete(): boolean
+ getEndTurnResult(): EndTurnResult
+ static success(payload: T, endTurn: EndTurnResult): ActionResult
+ static fail(): ActionResult
<<enumeration>> ActionStatus
SUCCESS
INVALID_CARD_CHOICE
CONTINUE_ACTION
ROUTE_CLAIMED
INSUFFICIENT_CARDS
EndTurnResult
- outcome: EndTurnOutcome
- nextPlayer: Player
- finalScores: Map<Player, Integer>
+ static nextTurn(nextPlayer: Player): EndTurnResult
+ static lastRoundTriggered(nextPlayer: Player): EndTurnResult
+ static gameOver(scores: Map<Player, Integer>, winner: Player): EndTurnResult
+ getOutcome(): EndTurnOutcome
<<enumeration>> EndTurnOutcome
NEXT_PLAYER_TURN
LAST_ROUND_TRIGGERED
LAST_ROUND_CONTINUES
GAME_OVER